# Tech for Endangered Languages:

**Ben Jupina and Kendal Kalanish and Zhen-Yu Yang**
Department of Computer Science
University of Pittsburgh
{bjj35, kck43, zhy75}@pitt.edu

## Abstract

Large Language Models have emerged as tools in the preservation of endangered languages. This project examines the efficiency of various approaches to prompting LLMs with the task of labeling Taiwanese-Hokkien characters in a code-mixed Mandarin sentence. This paper also discusses the ethics and limitations of these technologies and suggests future work.

## 1 Introduction

Hokkien is a language commonly used in East and Southeast Asian regions, including parts of Mayalsia, Singapore, Taiwan, Fujian Province in China, and the Phillipines. There are many different dialects in the Hokkien diasphora, each with its own tonal preferences and words, but one of the more formally used is Taiwanse-Hokkien. Taiwanse-Hokkien is mainly spoken in Taiwan since it's one of their national languages. However, its use can be found in many surrounding areas such as Fujian and Hong Kong. With millions of speakers, Taiwanse Hokkien is the biggest dialect group in the Hokkien language family.

This being said, because of the fact that Taiwanse-Hokkien does not have a formal writing system, instead it has 4 writing systems, each used in different scenarios, there has been a surprisingly low resource on this language. The dwindling use of this language generation after generation poses a serious concern to the Taiwanse government for the death of this language. Baby boomers know about 95% of the language, while Gen X knows only about 70% to 80%, and less than a quarter of the Gen Z population can speak Hokkien well. In an effort to preserve and revive this dialect, Taiwanse government has shown significant effort in collecting and curating Taiwanse-Hokkien corpus for NLP purposes, albeit, most of its resources is hidden behind a paywall and inaccessible to the NLP community at large.

Nowadays, most of the Taiwanse-Hokkien is spoken in a mixed language settings. The younger generations tend to use Taiwanse-Hokkien as the embedding language and Mandarin-Chinese as the matrix language in their daily lives, while the older generations do the opposite. Therefore, a study into the code-mixed setting is especially important to understand how Hokkien is used in Taiwanese community and how to best preserve it. NLP tools already tend to struggle to process code-switched data, and linguists are commonly forced to annotate such data manually. Code-switched data where one of the language doesn't have a formal writing system, such as Mandarin-Chinese mixed with Taiwanse-Hokkien poses an interesting challenge with its lack of resource. And building a tool that can accurately identify the code-switched data in this setting could proved to be beneficial in many different tasks such as translating code-mixed sentences into monolingual sentenes, annotating code-mixed speech data, and facilitate Mandarin-only readers to read Taiwanese literature with many code-switched sentences.

## 2 Data

Since there is little to no data on code-mixed data in Mandarin-Chinese and Taiwanse-Hokkien, we have to resort to a synthetic solution for creating training datasets. In this project, we primarily looked at the data synthesized by Lu et al. (2022). They first standardized all of the Taiwanse-Hokkien corpus that have parallel translation in Mandarin-Chinese into one writing system, then used linguistic toolkit to efficiently segment the words into corresponding translations, and finally, generate datasets by sampling Taiwanse Hokkien sentences and randomly swap out words with their corresponding Mandarin-Chinese counterparts, yielding a constructed code-mixed dataset. Taiwanse-Hokkien characters are tagged with a tailing symbol _@, while the Mandarin-Chinese Han characters remain the same.

They empirically tested the efficacy of such technique by training a cross-lingual language model via transfer learning and evaluated its performance on translating from code-mixed data to monolingual data. However, after a manual review of their dataset, we've discovered that the majority of the Taiwanse-Hokkien characters are tagged as Mandarin-Chinese and vice versa, yielding a poor quality ground truth training set for token-level language identification. Nonetheless, it is the best resource we have on our hands.

In this project, we aim to expand on Lu et al. (2022)'s work and further test the robustness of their technique by evaluating their curated code-mixed datasets on the performance of token-level language identification task. We introduce a small high quality human curated code-mixed dataset to be used as benchmarking dataset to test the performance of a model's ability to correctly identify languages at the token level. The dataset is created and manually reviewed by multiple native Taiwanese-Hokkien and Mandarin Chinese speakers.

## 3 Methods

We can separate our approaches into 3 ways of using a large language model:

1. Zero-Shot prompting
2. Few-Shot prompting
3. Fine-Tuning LLMs using the OpenAI API

Each of these approaches was applied to our Evaluate data set and then analyzed for their efficiency. Below, we will discuss the reasoning and implementation of each strategy

### Zero-Shot

Zero-shot prompting has become ubiquitous in the world of ChatGPT, without many people realizing it. The zero shot prompt is a prompt where the prompts are not labeled and no examples are given to the AI model. An example prompt used in this strategy is:

```
You are an excellent linguist in Mandarin Chinese and Taiwanese Hokkien. Your task is to label the characters that are in Mandarin Chinese in the given Taiwanese-Hokkien sentence. Return only the sentence, with each Thaiwanese-Hokkien word labelled as @@<Thaiwanese-Hokkien word>##. 人肉鹹鹹的啦！你要把我怎樣？
```

Our prompt was written to give a LLM enough context to understand the task, but is still zero-shot since no examples are given. Our reasoning for including a zero-shot prompt was to use it as a baseline for comparison against our prompt-engineered and fine-tuned methods.

### Few-Shot

Few-shot prompting is a form of prompt engineering that allows model output quality to significantly increase without changing any model parameters. Few-shot prompting achieves this improvement through in-context learning, in which a LLM is able to predict better tokens and reduce loss during the forward-pass at inference time. An example prompt used in this strategy is:

```
You are an excellent linguist in Mandarin Chinese and Taiwanese Hokkien. Your task is to label the characters that are in Mandarin Chinese in the given Taiwanese-Hokkien sentence. Below are some examples.
  input: 請上門的客人試食
output: 請上門的@@客##@@人# #試食
  input: 想欲的不巧一公道佮尊嚴
output: 想欲的@@不##@@巧##一公道佮尊嚴
  input: 伊的修養真好，攏袂佮人冤家。
output: 伊的@@修##@@養##真好，攏袂佮人冤家。
input:  人肉鹹鹹的啦！你要把我怎樣？
output:
```

Our prompt is few-shot because it gives a few examples of what we want the output to be. We used this approach since it is a very low effort way to increase quality of results. It is a practical way to solve many problems using LLMs, since there is no associated training cost with few-shot prompting.

### Fine-Tuning

Fine-tuning is very different from the other approaches, since it involves training a LLM with more data, causing model parameters to change. Fine-tuning is typically used when the task is very specific or requires unique data. Our task was a good candidate for using fine tuning since there are very few written resources on code-mixed Mandarin-Hokkien. We used the OpenAI fine-tuning API which, which is trained using prompt response examples in a jsonl file, for example

```
"messages": ["role": "user", "content": "加上肉仁卵白質含量懸，", "role": "assistant", "content": "加上肉仁@@卵##白質@@含##@@量##@@懸##，"]
```

We used over 7500 data points similar to the one above for our fine tuning model. Prompting the fine-tuned model is simple, since we just prompted with a non delimited code-mixed sentence. We used this approach since we thought that generic LLMs would struggle with our task without being exposed to additional Mandarin-Hokkien code-mixed data, due to a lack of initial training data.

## 4 Results

We evaluated the model using accuracy, precision, recall, and the F1 score. Accuracy measures how many of the tokens were labeled correctly out of all the predictions. Precision measures how many times the model correctly labels a token as Hokkien. Recall then measures how many tokens the model labels as Hokkien out of all of those that should be labeled Hokkien. These metrics allowed us to compare the different approaches to prompting the model in a quantitative way. This part of the project was done using Python.

This task first required the reformatting of the output of the model. This required us to parse the results of the model to only get the text that was going to be compared. We decided to keep the formatting of the original data for evaluation, so each token that was labeled as Hokkien is surrounded by "@@##".

We then had to align the two reformatted strings in order to compare them. This was done using a module from the Python standard library called difflib. We used this to calculate the difference between the two strings, both formatted, and then align them, putting a '-' in places where there was an insertion or deletion to get them to the same length.

Finally, we utilized the scikit-learn python library to compute the metrics of the reformatted and aligned strings. Table 1 contains the accuracy, precision, recall, and F1 scores for each version of our testing, including the performance of zero-shot prompts, few-shot prompts, and prompting a fine-tuned model. As is seen in Table 1, the fine-tuned model performed best across all metrics, and the model given zero-shot prompts performed the worst.

| | Zero-shot | Few-shot | Fine tuned |
|---|---|---|---|
| Accuracy | 0.3687 | 0.4350 | 0.4406 |
| Precision | 0.3447 | 0.3810 | 0.3963 |
| Recall | 0.3302 | 0.3539 | 0.3950 |
| F1 | 0.3409 | 0.3149 | 0.3589 |

Table 1: Caption

## 5 Discussion

The results of our project are quite consistent. This can be attributed to our small amount of data. The worst performance was of the zero-shot prompts. This was due to the lack of examples or further information given to the model, resulting in inconsistent results. This also reflects the possible small amount of code-mixed Taiwanese-Hokkien and Mandarin Chinese data that is used to train the model tested. Fine-tuning, even with little data, was able to improve the performance of the model.

## 6 Future Work

This project can be expanded upon by finding, processing, and testing more data. We were limited by the small amount of real-world data examples since the only large corpus we had was synthetically generated. Having better data could be more indicative of how the code mixed language is used in real life, and would allow us to make a more accurate and useful classifier.

We only investigated OpenAI technologies for this project, but it would be interesting to explore other large language models as well as other NLP systems. Other companies, such as Meta's Llama model, have been used to explore Sino-Tibetan languages, so it would be an interesting model to use in the future. There are other, older language models that could be used that are typically used for sequence labelling that could have interesting applications to our task, such as a BERT model.

## 7 Limitations

One of the most challenging limitations that we had to regularly deal with was that there are very few resources for Mandarin-Hokkien code-mixed data. 95% of Taiwanese baby boomers are fluent in Hokkien while less than 25% of Taiwanese Gen Z know the language.

This lack of data made it difficult to find large, relevant data sets to use for fine-tuning. To increase the amount of data that we had access to, we used data that was artificially synthesized rather than actually collected. We were thus faced with a trade off where we needed to use artificially collected data to increase the quality of our results, with the risk of using data that is potentially not reflective of how the code-mixed language is typically used by its speakers.

On a technical side, the OpenAI API was difficult to use at times due to it being closed source. Especially when fine-tuning a model, it felt difficult to know if improvements were actually being made. It was also frustrating how expensive fine-tuning was.

## 8 Ethical Issues

Taiwan, the country where this code-mixed language is spoken, has been very important politically for the last few decades. Its independence is not recognized by all nations, specifically China, where Mandarin is the official language. The political tensions caused by this geopolitical instability has a great influence on the code-mixed language; there have been efforts in Taiwan to "de-Sinicize" Taiwan by educating a larger amount of the population of Taiwan in "mother-tongue" languages like Hokkien.

The work that we are doing could be seen as politically sensitive, since it explores two languages that have a very unique power dynamic. There are many pro Taiwanese independence people that want to move away from Mandarin, and there are many pro China people that want to continue using Mandarin, and our project is in the middle of two competing ideologies that use language as a political tool. To further complicate things, Hokkien and Mandarin are very similar languages, so it is sometimes difficult to definitively decide which words come from which languages.

The very political nature of these languages, and the similarity of them thus give a classifier large ethical implications. Some could view our classifier as supporting one side of the Taiwan political conflict, and thus contributing to Mandarin dominance over Taiwan, which would be seen as a very political stance. It is difficult to design a piece of technology that uses data in a very politically sensitive area without implicitly contributing to either side of the conflict.(Huang, 2020)

## 9 Group Member Task Breakdown

### Ben

Ben was responsible for the OpenAI API call parts of the project. He data wrangled data sets that Zhen-Yu collected so that they were the correct format for the API. He then used the OpenAI API to collect and organize responses. He also used the OpenAI API to create the fine-tuned model.

### Kendal

Kendal was responsible for the evaluation part of the project. She parsed and formatted the outputs of the models and used them to generate the metrics used in the results section.

### Zhen-Yu

Zhen-Yu was responsible for the data collection part of this project. He helped collect the necessary training data and manually created the evaluation dataset for benchmarking. He also helped with preprocessing and formatting the data.

## References


Shuanfan Huang. 2020. Language policy and practice in taiwan in the early twenty-first century. *Language Policy*.

Sin-En Lu, Bo-Han Lu, Chao-Yi Lu, and Richard Tzong-Han Tsai. 2022. Exploring methods for building dialects-Mandarin code-mixing corpora: A case study in Taiwanese hokkien. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 6287–6305, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.